
电商网站虚假网购评论识别

首都经济贸易大学

王禹、张世平、戴德

摘要

随着互联网热潮兴起,网购逐渐成为消费者购物途径的一个重要选择,当下电商平台众多,商品种类齐全,样式琳琅满目,消费者的选择实现了多样化,而消费者对是否选择购买电商网站的某些商品的一大依据就是商品的评价信息。良好的商品评价会使得消费者更愿意购买商品,由此,一些商家就会通过各种途径伪造评论,这无疑会对消费者的选择与判断形成干扰。本文使用网络爬虫技术获取样本商品,利用文本挖掘与机器学习技术进行文本分类,进行虚假评论的识别。即通过样本商品,构建一个扩展性较强的虚假网购评论识别的流程与体系,进行虚假评论的甄别,从而对商品评论的真实性给出较可观的评价。

关键词:虚假评论 文本挖掘 机器学习

1 问题背景及现状

近年来，电子商务由于其市场全球化成本低廉化、资源集约化等优势，迅速成为了全球消费市场的主流，逐渐深入到我们生活的方方面面。同时中国电子商务行业也在迅猛地发展，不断有新的公司加入电子商务行业，电商平台数量也迅猛增加，电商平台的规模越来越大，电商平台包含的商品种类也越来越多。既有京东、天猫这样超大电商平台，也有主要经营 3C 和家用电器的电商平台，还有针对海外代购的电商平台，以及针对高端消费者的奢侈品电商平台。此外，每个电商平台还在不断地丰富自己的业务，扩大自己的板块。

电商的兴起也为更多的中小型企业提供了平台，电商平台为他们提供了一个销售渠道和广告平台。人们越来越多地依赖互联网资源来为自己提供决策支持，消费者在购买商品之前往往首先查看该商品的历史评论，更多的正面评论能够促使消费者购买该商品，而更多的负面评论则会使消费者放弃该商品。评论的好坏和质量很大程度上确定了一个新的消费者会不会尝试购买这个产品，老用户是否会继续选择购买这个产品。因此，用户的评论自然成为了重要的参考标准。不仅仅是对于消费者来说，对于电商平台和商户来说也是非常重要的。用户的评论反映了消费者的消费习惯，以及对产品的反馈，这些都是非常重要的资源，帮助商户进行提升和改善。

一些问题也随之而来，也就是评论的真实性。一些商户在最初解

读很难去吸引顾客，因为在搜索结果前列的都是销售量高或者好评数量多以及综合排名高的商品。为了吸引顾客和抢夺市场，许多的商家会选择刷评论的方法，获得数量多且评价高的评论。商户可能或选择获得自己的家人或者朋友，通过他们购买商品从而快速获得好评，但是这种获得的评论数量较少效率比较低。这种需求也就促成了刷评论的产业链的生成。会有专门的团队来负责这种类型的工作，首先会在贴吧、论坛、微博、QQ 和微信等平台招募人员，还会有培训如何减少被发现刷评论的概率。还会有专门负责接刷单任务的人员，分配刷单任务的人员。形成了有规模有组织的团体。

为了了解“专业刷手”是如何工作的，我们缴纳会员费进入刷单组织调查其刷单模式。目前的刷单非常专业，从电商平台内部识别恐怕都非常困难。刷手抢到单以后，按照要求，进行操作，一般会要求刷手搜索一个词条，浏览词条下若干家店铺，每个店铺至少保证浏览若干分钟，然后进入指定店铺，从最顶部浏览到最底部，重复三次，要求一定浏览时间，更有甚者需要和客服进行交谈，最后下单（分为商家垫付和刷手自行垫付，自行垫付佣金更高），然后指定时间后收货（并没有真实货物，但是店铺确实可以给出发货信息，如订单号），对商品评论，但商品评论的内容无人指定，由刷手自己完成。低价商品（200 元以下）刷手的佣金基本在 0.3-1 元。

随着日常生活的数据量的爆炸式增长，需要通过诸多的技术来发掘蕴藏在这些数据中有价值的信息。非结构化的数据越来越重要，文

字评论是非结构化数据的一类。因此文本挖掘作为大数据分析领域中的一个分支，也发挥着越来越重要的作用。人工从成千上万的评论中分辨出虚假评论是一项非常庞大的任务，运用机器学习的方法可以解决这个问题，并且能够大幅的提升工作的效率。

在刷评论的商品类别中，刚需较强的商品和高价商品（如数码电器类）刷评论的比例较低，而低价、快消品（如服装、鞋帽类）刷评论的比例较高。本文选择了女鞋品牌的评论，从以下两点考虑：首先考虑到了市场的需求量，相对于男性的服装鞋类商品，女性的服装和鞋子更像是一类快消品，具有很高的需求量和较短的更替周期。女性在购买商品时会更多的考虑评论的内容，并且她们也会更多的倾向于发表评论，评论的内容和样式也会更加的丰富。

2 国内外研究情况

美国伊利诺斯州大学芝加哥分校教授 Bing Liu^[1]及其团队于 2007 年首次提出了在线商品虚假评论检测的相关研究问题，他将商品虚假评论分为三类：⌘

1. 欺骗性评论：是指那些不是基于评论者真实的产品或服务体验，而蓄意吹捧或贬低产品声誉的欺骗性评论。
2. 不相关的评论：是指评论的对象仅仅是关于品牌、生产商、⌘销售商等其它与产品本身无关的评论。
3. 非评论信息：该类型信息主要包括两种类型；广告和不表达观点的

文本（如读者的问题、回复和随机文本）。

后两种类型的垃圾评论所占的比重相对较小，且特征非常明显、易于区分，使用一般的监督学习的方法就能够很好地将其识别出来。而且，人能够很容易地识别出这两种类型的评论，在实际生活中这些评论也很难对消费者造成误导。目前，关于虚假评论检测的研究主要是针对第一种类型的欺骗性评论检测。

Feng^[2]等使用新颖的基于概率的上下文无关语法规则特征，用 SVM 分类器进行真实与虚假文本分类，并在金标准数据集上获得了一定分类率的验证。评论内容的真实和虚假辨别，涉及评论文本的自然语言理解、意见抽取过程，因此基于评论内容的虚假评论检测比较困难，并且效果不是很好。于是，相关学者开始基于评论者行为对虚假评论的检测开展相关研究。

国内研究领域，任亚峰^[3]等人通过 PU (positive and unlabeled) 算法，通过少量的真是评论和大量的未标注评论建立合适的分类器。在此基础上通过基于狄利克雷过程混合模型的非参数贝叶斯聚类方法，并且使用多核的 SVM 方法，提升了分类效果。赵军^[4]等人提出了基于文本的情感分析通过 Logistic 回归模型识别虚假评论，根据程度副词的情感程度的情感程度分别赋予不同的权重。然后再根据关联词前后的情感极性进行分类，最后根据 Logistics 模型对评论进行分类。

3 方案设计

整个专业刷评论流程比较严谨,从外部也无法获取更多订单数据,但是刷手最终的评论是自由撰写的,毕竟,刷手从未接到过商品,也没有对商品切实使用,因此,由评论本身而推究评论的真实虚假是我们的总体思想。

本文采取的方案如图 3.1 所示,确定好目标商品后,进行其评论时间及评论内容的获取,通过对评论时间的分布情况对评论的真假进行“预识别”(对于本文所选取的样本,评论时间不存在异常)。

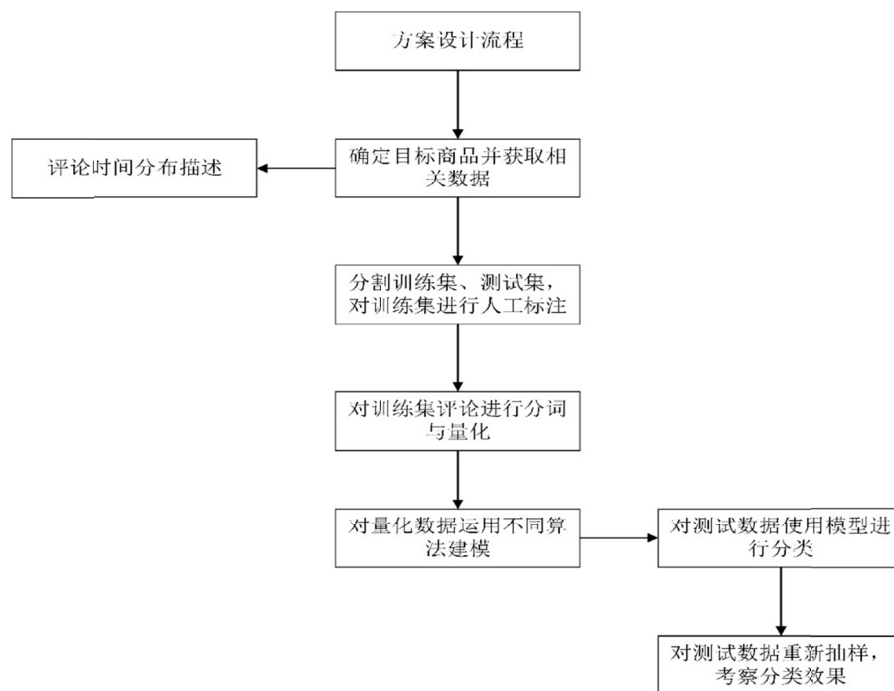


图 3.1 方案设计流程

对所获取的样本,分为训练集和测试集,我们制定了一套针对本类商品的虚假评论识别标准,对训练集样本进行人工标注,标注其属于“真实评论”还是“虚假评论”(1为虚假评论,0为真实评论,我

们的目标是更好的识别出 1 类)。然后对全部评论内容进行分词与量化, 再对量化后的训练集数据采用不同机器学习算法(非线性可分支支持向量机、朴素贝叶斯、深度神经网络以及组合模型)进行建模、分类, 得到评论属于“真实”还是“虚假”, 以及评论属于虚假评论的概率。再将我们的模型用于测试集数据, 对测试数据进行分类。最后从测试数据中进行抽样, 重新依照判别标准进行人工标注, 计算其准确率、召回率、F 得分等, 考察我们模型的效果。

之所以要将评论的类别和其属于虚假类别的概率都给出是为了使分类结果更“柔软”, 不将分为“虚假”类别的评论“一棒打死”, 而给出其属于虚假评论的概率, 可以让本文所设计的虚假评论流程的扩展性更强。

4 数据说明

4.1 数据获取

在诸多电商平台商品中, 我们选取“女鞋”作为我们的目标数据, 原因是服装、鞋帽类商品刷评论现象较严重(如第 1 章所述)。

本文数据来源是天猫商城, 样本品牌为女鞋“珂卡芙”, 数据来自于天猫商城三家不同店铺(非官方旗舰店)的同款女鞋。鉴于对商铺的保护, 本文不展示任何有关店铺的信息。

本文使用 Python3 进行数据爬取。天猫商城商品评论数据属于动态存储方式, 存储与 json 中, 本文利用 Python 的相关库, 获取存储

商品评论的 json，并以解析 json 的方式获取评论数据。三家商铺数据共 4140 条，样本包含四个变量：用户昵称、用户等级、评论时间、评论内容。由于用户昵称对我们的分析过程基本没有贡献，因此予以忽略，用户等级中，超过 90% 的用户属于“无评级”，同样予以忽略，我们使用评论时间和评论内容进行后续分析。

评论时间可以通过其分布考察评论者的评论时间是否存在异常（如是否存在某个时间段中出现大量评论），如有异常，则可以做到“预识别”的效果。而评论内容是我们研究的重点，我们将通过文本分析的技术进行虚假评论的甄别。

4.2 数据处理

对于这 4140 条评论，我们抽取了其中 1000 条作为待标注样本，本小组三名成员独立对其进行“真实评论 vs 虚假评论”标注，1 为虚假评论，0 为真实评论。对于鞋类商品，用户最关注的就是鞋子本身的舒适度，其次就是鞋子的细节、质量、外观，故我们将标注的基本规则指定为：评论中鞋本身质量、舒适度、外观等为主的（对于本款商品，很多评论会谈及鞋子是否打脚、磨脚，鞋跟是否舒适，鞋带等质量和舒适度问题）则认为评论为真实评论，评论中谈及“还未试穿，还未上脚，穿后追评”等内容的认为是真实评论，评论内容越细致、越多描述细节的则认为是真实评论。若评论以描述商家、客服态度、物流为主则认为是虚假评论。总结标注规则，基本可以概括为如下几条：

针对本商品：

- 绝大多数评论为好评，不过其中很多提及了鞋子味道大，绑带松，鞋跟做工有些粗糙，号码问题等，涉及这些内容的评论一般可以说明评论者真实收货并试穿，认为基本是真实评论。
- 以及对于鞋类的消费者而言，鞋子的质量、舒适程度基本上是最重要的，其次是外观等，所以以质量、细节为主的评论认为是真实评论。
- 不涉及质量的评论，或者不含以上内容的过短评论认为是虚假评论（“用户未在规定时间内评论，默认好评”标注为真实评论）。
- 一些评论中含有“还未穿，穿后追评”一类信息，这类评论认为是真实评论。
- 评论中不提及鞋子本身质量和细节，只有商家、客服态度、物流等内容的评论认为是虚假评论。

标注规则由本小组三人独立标注，最后进行投票，对于每条待标注样本，我们取三个标注结果的众数，防止由一个人标注而引起的过强主观性。

在人工标注的 1000 条评论中，标注为虚假评论和真实评论的比例约为 1：2，为了优化模型，提高预测精度，我们对虚假评论采取了“过抽样”，来解决不平衡分类问题，优化该流程。

从虚假评论中找出几个代表评论进行展示：

- 鞋子收到了，物流很快，昨天下的单今天就到了，穿了，效果很好，好评支持一下
- 物流发货快，客服态度好，非常喜欢。
- 好评，大牌
- 鞋子很不错的，发货也非常的快，满意的。
- 宝贝收到了，好评，物流很快，是正品，下次会再来这家店购买的，超级

满意。

- 物流很快 鞋子很好 客服很萌
- 鞋子非常棒，很满意，客服服务也比较好，很满意
- 快递挺快的，质量可以，卖家态度不错
- 鞋子很满意，颜色很美，快递很给力，么么哒

本文的目标是建立虚假评论的识别流程与方法，在算法上使用的是非常科学、客观的方法，但在标注方面比较主观，是通过咨询一些女性消费者，了解其对鞋类商品进行评论时的关注点，从而建立了标注规则。对于不同商品，必然有不同的规则，只要建立了一套标注规则，运用本文的流程就可以进行虚假评论的识别。

5 虚假评论识别

5.1 评论时间分布

三家店铺的商品评论时间如图 5.1 所示。从评论时间来看，评论时间主要分布在 10 点到 22 点，比较均匀，并未出现明显的“尖峰”现象，也就是说，没有明显的集中评论时间（由于每天的评论数量较少，故未展示每天的评论数量，也不存在集中评论现象）。如果查看评论时间时，发现存在集中评论时间，那么处于集中评论时间中的评论，可能存在大量的虚假评论，应当在标注时予以注意，除明显的真实评论，其余评论应当视为虚假评论。

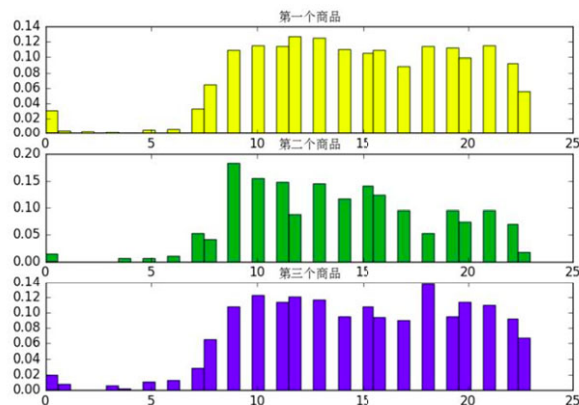


图 5.1 评论时间分布

5.2 商品评论词云图

图 5.2、图 5.3 和图 5.4 分别为用户总体评论词云图、标注为真实评论的词云图和标注为虚假评论的词云图。

可以看到真实评论和虚假评论中“喜欢”、“不错”、“好看”等词语出现频率均比较高,这是由于评论中绝大部分为好评。观察细节可以发现,真实评论中的高频词汇还有“质量”、“磨脚”、“穿着”、“舒服”等涉及质量和细节的词汇,而标注为虚假评论的评论中高频词汇主要都是一些比较空泛的情感词汇。

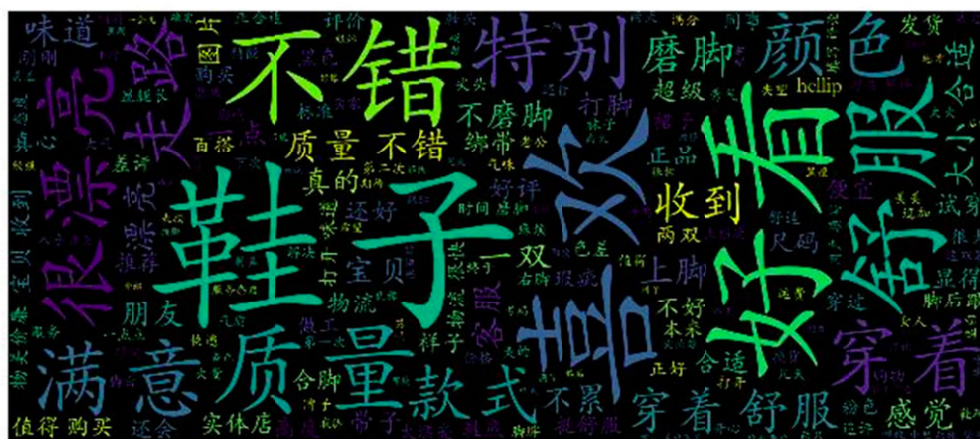


图 5.2 用户总体评论词云图



图 5.3 标注为真实评论的词云图



图 5.4 标注为虚假评论的词云图

5.3 文本分类

5.3.1 文本分类原理

文本数据属于非结构数据，对于非结构数据，首先要将其转化为结构数据，就是使用数值对文本信息进行表示。

首先应当对数据进行分割，分为训练集和测试集，在训练集中训练模型，优化参数，在测试集上使用模型，并作出评价。其评价方法和结构化数据分类的评价方式相同，也使用准确率、召回率、F1 得

分等指标进行评价。

文本分析的第一步一般是分词。分词技术经历过几次变革，目前的分词技术已经非常成熟，可以达到比较高的准确率。本文使用 Python 的 “jieba” 库进行分词处理，分词后需要去除停用词。

接下来，要对分词后的文本信息进行量化。文本信息量化的方式有很多，如常用的 TF-IDF 方法、哈希向量法等，本文采取哈希向量法。

对于结构化的数据，就可以正常的使用各种分类算法，文本分类中常用的方法有：朴素贝叶斯、支持向量机等方法，本文采取了这两种算法，并使用了深度神经网络算法以及组合算法（前三种算法的组合）。

最后，对模型的分类效果进行评价，考察其准确率、召回率、F1 得分指标。

5.3.2 非结构化数据结构化处理

首先要对分词后的非结构化数据进行特征向量的提取。中文词转特征向量方法有多种，如 TF-IDF 方法。这里我们使用另一种方法：哈希向量法。哈希向量法在 Python 中调用 HashingVectorizer 方法，其中的 `n_features` 参数定义了所转成特征向量的维度大小，或者说属性多少。维度越大，代表描述其本身的属性越多，结果就会越精确，但是这样也就意味着计算和存储等消耗也就越大，需要在此间折衷抉

择，依据不同的情况，选择适当的 `n_features` 参数大小，若不特殊指定 `n_features` 参数大小，那么它的默认值是 2 的 20 次方。这里，考虑到计算资源以及文本本身的长度，我们将维度定为 500。

5.3.3 基于机器学习的文本分类

本文使用了 SVM、朴素贝叶斯、深度神经网络和组合算法，对训练集的评论进行模型建立。将训练集的 70% 用于模型训练，剩余 30% 作为验证集，进行模型效果考察和参数调优。综合考虑在各个参数下模型在训练集和验证集的效果，来确定最优参数。这里模型效果考察使用准确率进行度量。

一、支持向量机（SVM）

支持向量机（SVM）是 90 年代中期发展起来的基于统计学习理论的一种机器学习方法，通过寻求结构化风险最小来提高学习机泛化能力，实现经验风险和置信范围的最小化，从而达到在统计样本量较少的情况下，亦能获得良好统计规律的目的。

通俗来讲，它是一种二类分类模型，其基本模型定义为特征空间上的间隔最大的线性分类器，即支持向量机的学习策略便是间隔最大化，最终可转化为一个凸二次规划问题的求解。

支持向量机分为线性可分、线性不可分和非线性可分三种类型。线性可分可以理解为用一条直线或一个超平面即可将数据完美的分割为目标类别；线性不可分指的是用一条直线或一个超平面无法将数

据完全分割，需要引入松弛变量，使得直线或超平面尽可能好的将数据进行分割；而非线性支持向量机是指无法使用直线或超平面对数据分类，这种情况下，一般需要将原始数据映射到高维空间，使得在高维空间中可以用线性决策边界对数据进行分类。

本文使用的是非线性可分支持向量机，核函数采用 sigmoid 核。不同参数形成的模型在训练集和验证集上的效果如图 5.5 所示。

结合模型效果图和每次参数调整时测试集的混淆矩阵，最终将最优参数 γ 定为 0.449843。

可以看到，SVM 在验证集上对虚假评论识别的准确率约为 70%，召回率为 83%，F1 得分为 0.75，效果比较好。

表 5.1 SVM 在验证集上的混淆矩阵

svm	precision	recall	f1-score	support
0	0.8	0.66	0.72	207
1	0.69	0.83	0.75	192
avg/total	0.75	0.74	0.74	399

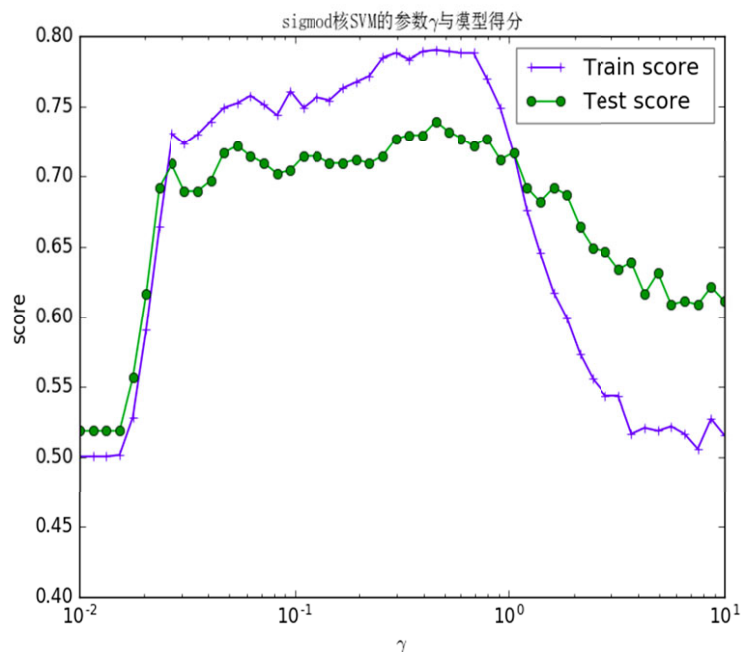


图 5.5 SVM 参数调整效果

二、朴素贝叶斯

贝叶斯分类是一类分类算法的总称，这类算法均以贝叶斯定理为基础，故统称为贝叶斯分类。朴素贝叶斯分类是一种十分简单的分类算法，朴素贝叶斯的思想基础是这样的：对于给出的待分类项，求解在此项出现的条件下各个类别出现的概率，哪个最大，就认为此待分类项属于哪类。其计算的就是如下公式所示的概率。

$$P(y_i|x) = \frac{P(x|y_i)P(y_i)}{P(x)}$$

贝叶斯分类也是文本分类中的一个常用算法，其在训练集和验证集上的效果如图 5.6 所示，在测试集上的混淆矩阵如表 5.2 所示。

将该算法运行在训练集和验证集上，发现参数的改变基本不影响测试集和验证集的分类效果（准确率），该模型在验证集上对虚假评

论分类的准确率约为 70%，召回率为 71%，F1 得分为 0.7。效果与 SVM 相近。

表 5.2 朴素贝叶斯在验证集上的混淆矩阵

nb	precision	recall	f1-score	support
0	0.73	0.71	0.72	207
1	0.69	0.71	0.7	192
avg/total	0.71	0.71	0.71	399



图 5.6 朴素贝叶斯参数调整效果

三、深度神经网络（DNN）

所谓的深度神经网络 DNN，往往融合了多种已知的结构，包括卷积层或是 LSTM 单元。但是就题主的意思来看，这里的 DNN 应该特指全连接的神经元结构，并不包含卷积单元或是时间上的关联。

单从结构上来说，全连接的 DNN 和上图的多层感知机是没有任何区别的。连接 DNN 的结构里下层神经元和所有上层神经元都能够形成连接，带来的潜在问题是参数数量的膨胀。

深度神经网络的结构可以简单用如图 5.7 的形式表示。

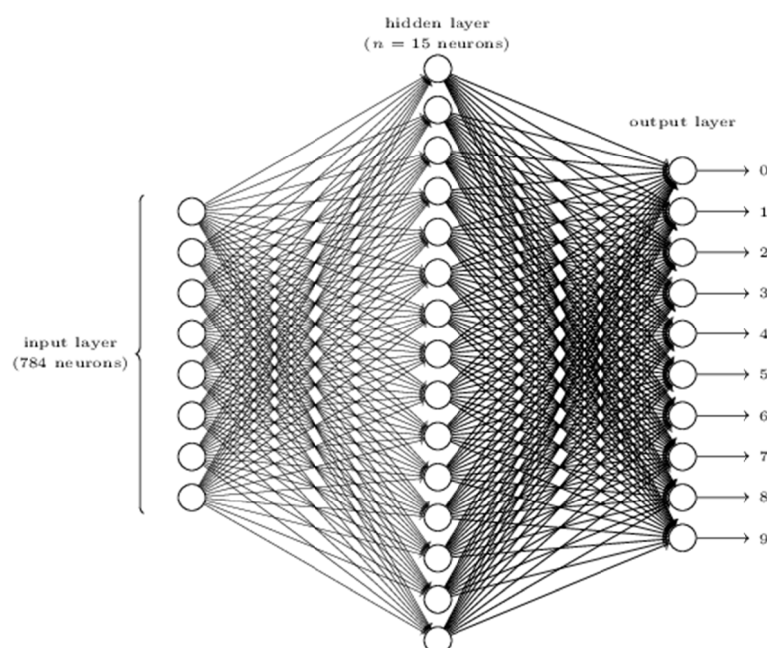


图 5.7 深度神经网络结构

深度神经网络（DNN）使用网络中层内神经元相互链接，层间神经元相互链接的方式，即全链接模式实现对数据的良好拟合，由于该方法的时间复杂度较高，且容易过拟合，所以对网络隐藏层数量、各层结点数的设置应当进行比较谨慎的确定，而且该方法用于大规模数据时比较耗费时间，需要分布式计算框架辅助实现。

本文通过不同参数的 DNN 模型在训练集和验证集上的效果来进行参数调优。结合每次参数调整时测试集的混淆矩阵，最终设置了 3 个隐藏层，各层结点数分别为 (25, 30, 25)。模型在训练集和验证

集上的效果如图 5.8 所示。最终模型在验证集上的混淆矩阵如表 5.3 所示。

DNN 算法对于虚假评论识别的准确率约为 80%, 召回率为 87%, F1 得分为 0.83, 要优于前两种方法。效果非常不错。

表 5.3 DNN 在验证集上的混淆矩阵

dnn	precision	recall	f1-score	support
0	0.87	0.79	0.83	207
1	0.79	0.87	0.83	192
avg/total	0.83	0.83	0.83	399

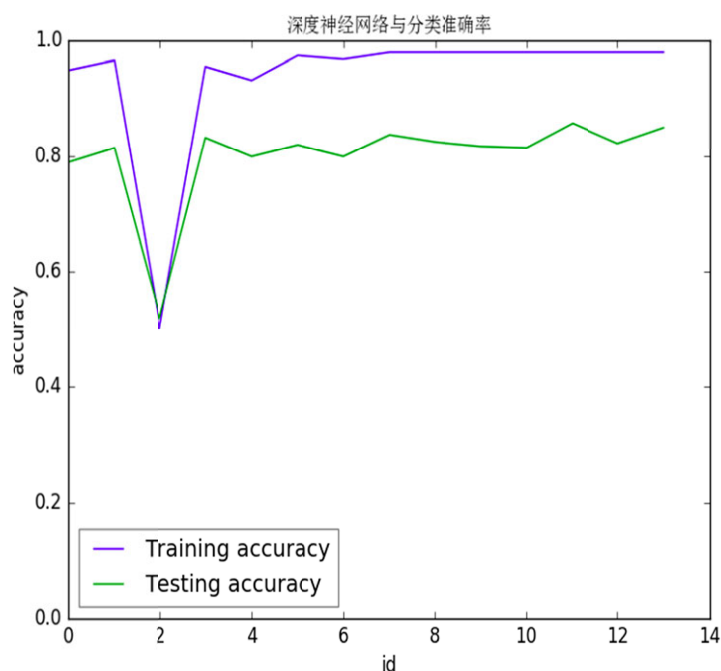


图 5.8 深度神经网络参数调整效果

我们使用三种方法对未标注的 3140 条评论进行预测，并对预测结果进行“投票”，得到了最终的预测结果。

预测显示，在测试集中，虚假评论占比 58%，真实评论占比 42%。

接下来，我们从测试集中抽取了两个样本，样本量均为 100，并

重新进行了标注，两次抽样结果的混淆矩阵结果相似，接下来进行展示（两次结果比较相近，故对其中之一进行展示）。如表 5.4 所示。对于虚假评论识别的准确率和召回率分别为 72%和 74%，F1 得分为 0.73，可见模型对于未知样本的分类效果是比较好的。

表 5.4 对测试集预测结果抽样检验

集成	precision	recall	f1-score
0	0.69	0.63	0.66
1	0.72	0.74	0.73
avg/total	0.71	0.69	0.69

至此为止，本文已经完成了对评论数据进行分类的目标。如本文第二章所述，本文认为，不仅应当对类别实现判断，还应当对每条评论属于两个类别的概率给出。这样可以使我们的结果更“柔软”，具有更强的扩展性与推广性。给出样本属于虚假评论以及真实评论的概率，可以通过修订阈值来重新进行分类，可以以不同的尺度去划分类别。

三种算法得到的分类概率结果分别如表 5.5-表 5.7 所示。由于算法原理不同，支持向量机和朴素贝叶斯算法对两个类别的分类概率都接近 0.5，而深度神经网络算法对两个类别的分类概率值存在显著差异，均接近 0 或 1，这也是深度神经网络算法在结果上优于另外两个方法的原因所在。

表 5.5 SVM 算法分类概率

真实评论概率	虚假评论概率
0.54200137	0.45799863
0.54552987	0.45447013
0.51156341	0.48843659
0.52467665	0.47532335
0.50869984	0.49130016
0.51454038	0.48545962
0.53218623	0.46781377
0.52918488	0.47081512
0.54349389	0.45650611

表 5.6 朴素贝叶斯算法分类概率

真实评论概率	虚假评论概率
0.50748605	0.49251395
0.50847972	0.49152028
0.49564132	0.50435868
0.50034429	0.49965571
0.49360634	0.50639366
0.49600577	0.50399423
0.49001773	0.50998227
0.50354711	0.49645289
0.50233676	0.49766324

表 5.7 DNN 算法分类概率

真实评论概率	虚假评论概率
1.00000000	0.00000000
0.99998522	0.00001481
1.00000000	0.00000000
0.99997807	0.00002197
0.00021303	0.99978703
0.00013560	0.99986434
0.00000051	0.99999952
0.00002456	0.99997544
0.99999738	0.00000258

6 结论

电商平台中商家的竞争日益激烈，若不依靠刷评论，商家可能很难在这个竞争激烈的环境中存活下去，尤其是对于新的店铺而言。雇佣刷手刷评论的成本极低，基本可以忽略，因此这是商家快速提升店铺热度的一个主要方式。虽然这是对消费者的一种信息不对等的不公平待遇，但是退一步讲，如果能够识别刷评论比例，在一定程度上可以让消费者更好的对商家和商品的质量作出判断。

本文提供了外部识别虚假评论的方法和流程，只要制定好针对目标商品虚假评论的识别规则，就可以完成虚假评论的识别，进而得到刷单比例。另外，评判刷单的方式可以不“一棒打死”，而以概率的形式给出，设定好一个阈值，可以统计虚假评论概率高于阈值的评论比例，也是一种可选的手段。

对于“识别规则”，也就是预先标注过程，是一个不可避免的人工、主观过程，当对于某类商品的数据量更大时，增加标注样本数量，增加标注结果（比如说对于鞋类商品，爬取更多评论，增加更多标注人员，得到更多的标注结果，然后对标注结果进行投票来确定最终标注），都可以在一定程度上优化标注过程。这个标注过程可以说是针对一类商品而非一个商品的。因为同一类商品的特征非常相近，多来源的评论数据也会使标注结果更加具有普遍性。

刷单恶性竞争严重，完全不刷好评的店家恐怕不多，不能说有刷

评论的店就完全不能下手，90%刷单的商品实在骇人听闻，10%刷单的店则或许质量尚可接受。这也进一步说明了我们的模型的作用：判断商品的刷单比例，比逐条判断评论是否虚假更加实用。

7 致谢

在这里，我们要感谢我们的指导老师阮敬老师，阮敬老师在我们的研究过程中提供给我们很大的指导，并给我们提出了宝贵的意见和建议，在思路、方法上、包括一些细节方面的建议都对我们有巨大的帮助。在这里我们要衷心感谢阮敬老师。

8 参考文献

- [1]Jindal N, Liu B. Analyzing and Detecting Review Spam[C]// Seventh IEEE International Conference on Data Mining. IEEE Computer Society, 2007:547-552.
- [2]Feng S, Banerjee R, Choi Y. Syntactic stylometry for deception detection[C]// Meeting of the Association for Computational Linguistics: Short Papers. Association for Computational Linguistics, 2012:171-175.
- [3]任亚峰, 姬东鸿, 张红斌,等. 基于 PU 学习算法的虚假评论识别研究[J]. 计算机研究与发展, 2015, 52(3):639-648.
- [4]赵军, 王红. 融合情感极性和逻辑回归的虚假评论检测方法[J]. 智能系统学报, 2016, 11(3):336-342.
- [5]张考. 面向电子商务的虚假评论检测的关键技术研究[D]. 解放军信息工程大学, 2015.
- [6]陈燕方, 娄策群. 在线商品虚假评论形成路径研究[J]. 现代情报, 2015, 35(1):49-53.
- [7]宋海霞, 严馨, 余正涛,等. 基于自适应聚类的虚假评论检测[J]. 南京大学学报(自然科学), 2013, 49(4):433-438.

9 附录

爬虫代码：

```
# -*- coding: utf-8 -*-
```

```
import urllib2
```

```
import json
```

```
import sys
```

```
import time
```

```
import re
```

```
import csv
```

```
# 指明解码方式为 utf8
```

```
reload(sys)
```

```
sys.setdefaultencoding('utf8')
```

```
csvfile = file('dqsm_gn1_tm.csv', 'rb')
```

```
reader = csv.reader(csvfile)
```

```
link = []
```

```
for line in reader:
```

```
link.append(line)

csvfile.close()

# 获取数据并解析

header = {'User-Agent': 'Mozilla/4.0 (compatible; MSIE 5.5; Windows
NT)'}

for j in range(len(link)):

    nickname = []

    level = []

    rateContent = []

    rateDate = []

    comment = []

    comment_json = []

    print '开始爬行商品%s 评论数据, 当前时间： %s' % (link[j][0],
time.asctime())

    start_time = time.time()

    try:
```

```
pattern = re.compile(r'&currentPage=1')

res = re.split(pattern, link[j][1])

url0 = res[0] + str('&currentPage=1') + res[1]

req0 = urllib2.Request(url0, headers=header)

cont0 = urllib2.urlopen(req0).read()

rex0 = re.compile(r'\w+[(\{1}(.*)[)]\{1}')

content0 = rex0.findall(cont0)[0]

try:

    con0 = json.loads(content0, "gbk")

    pageNum = con0['rateDetail']['paginator']['lastPage']

except:

    try:

        con0 = json.loads(content0, "GB2312")

        pageNum = con0['rateDetail']['paginator']['lastPage']

    except:

        try:

            con0 = json.loads(content0, "GB18030")
```

```
        pageNum =
con0['rateDetail']['paginator']['lastPage']

    except:

        print '首页抓取失败，默认爬取 99 页！'

        pageNum = 99

    for p in range(1, pageNum + 1):

        url = res[0] + str('&currentPage=') + str(p) + res[1]

        req = urllib2.Request(url, headers=header)

        cont = urllib2.urlopen(req).read()

        rex = re.compile(r'\w+([1]{1}(.*))1}')

        content = rex.findall(cont)[0]

        try:

            con = json.loads(content, "gbk")

        except:

            try:

                con = json.loads(content, "GB2312")

            except:

                try:
```

```
        con = json.loads(content, "GB18030")

    except:

        continue

    comment_json.append(content)

    comment_json.append('\n')

    print '正在爬取第%d 页的评论，共%d 页' % (p,
pageNum)

    for i in xrange(20):

        try:

            nickname =

con['rateDetail']['rateList'][i]["displayUserNick"]

        except:

            nickname = "

        try:

            level =

con['rateDetail']['rateList'][i]["attributesMap"]["tmall_vip_level"]

        except:

            level = "
```

```
        try:

            rateContent =

con['rateDetail']['rateList'][i]["rateContent"]

            rateContent = rateContent.replace("<b>", "")

        except:

            rateContent = "

        try:

            rateDate =

con['rateDetail']['rateList'][i]["rateDate"]

        except:

            rateDate = "

        comment.append(nickname)

        comment.append('|')

        comment.append(level)

        comment.append('|')

        comment.append(rateDate)

        comment.append('|')

        comment.append(rateContent)
```

```
        comment.append('\n')

    f0 = open(link[j][0] + '_json.txt', "w")

    for item in comment_json:

        f0.write(item)

    f0.close()

    f = open(link[j][0] + '.txt', "w")

    for item in comment:

        f.write(item)

    f.close()

except:

    print '爬行商品%s 评论数据失败, 请重试' % link[j][0]

end_time = time.time()

used_time = end_time - start_time


print '爬行商品%s 完毕, 当前时间 :%s, 共耗时%s' % (link[j][0],
time.asctime(), used_time)
```

```
print '-----'
```

文本分类代码：

```
import pandas as pd
```

```
import numpy as np
```

```
from sklearn.feature_extraction.text import HashingVectorizer
```

```
from sklearn import cross_validation, svm
```

```
from sklearn.metrics import classification_report
```

```
import matplotlib.pyplot as plt
```

```
from sklearn.naive_bayes import MultinomialNB
```

```
import tensorflow as ts
```

```
from matplotlib.font_manager import FontProperties
```

```
myfont = FontProperties(fname = r'C:/Windows/simsun.ttc')
```

```
data = pd.read_csv("result_use.csv")[0:1328]
```

```
# print(data.shape)
```

采用 HashingVectorizer 方法进行特征向量转化

vector = HashingVectorizer(n_features=500)

temp = vector.transform(data['comment']).todense()

print(temp)

y = data['target']

x = temp

查看数据 0-1 比例

print(sum(y))

x_train,x_test,y_train,y_test =

cross_validation.train_test_split(x,y,test_size=0.3,random_state = 1)

SVM

gammas = np.logspace(-2,1)

train_score = []

test_score = []

for gamma in gammas:

```
    clf = svm.SVC(kernel = 'sigmoid',gamma=gamma, coef0=0)
```

```
    clf.fit(x_train,y_train)
```

```
    print("gamma is : %f" % gamma)
```

```
    print(clf.score(x_train,y_train))
```

```
    train_score.append(clf.score(x_train,y_train))
```

```
    print(clf.score(x_test,y_test))
```

```
    test_score.append(clf.score(x_test,y_test))
```

```
    pred_test = clf.predict(x_test)
```

```
    print (classification_report(y_test, pred_test))
```

gamma 为 0.449843 最优

```
fig = plt.figure()
```

```
ax = fig.add_subplot(1,1,1)
```

```
ax.plot(gammas,train_score,label = "Train score ",marker = "+")
```

```
ax.plot(gammas,test_score,label = "Test score ",marker = "o")
```

```
ax.set_title(r"sigmod 核 SVM 的参数$\gamma$与模型得分
```

```
",fontproperties = myfont)
```

```
ax.set_xscale("log")
```

```
ax.set_xlabel(r"$\gamma$")
```

```
ax.set_ylabel("score")
```

```
ax.set_ylim(0.4,0.8)
```

```
ax.legend(loc = "best",framealpha = 0.5)
```

```
plt.show()
```

```
# MultinomialNB
```

```
for i in range(temp.shape[0]):
```

```
    temp[i] = temp[i] + 1
```

```
    y = data['target']
```

```
    x = temp
```

```
    x_train, x_test, y_train, y_test = cross_validation.train_test_split(x, y,  
test_size=0.3, random_state=1)
```

```
def MultinomialNB_alpha(*data):
```

```
    x_train, x_test, y_train, y_test = data
```

```
    alphas = np.logspace(-2,2,num=200)
```

```
    train_score = []
```

```
test_score = []

for alpha in alphas:

    cls = MultinomialNB(alpha=alpha)

    cls.fit(x_train,y_train)

    train_score.append(cls.score(x_train,y_train))

    test_score.append(cls.score(x_test,y_test))

    pred_test = cls.predict(x_test)

    print(classification_report(list(y_test), pred_test))

fig = plt.figure()

ax = fig.add_subplot(1,1,1)

ax.plot(alphas,train_score,label = "Training Score")

ax.plot(alphas,test_score,label = "Testing Score")

ax.set_xlabel(r"$\alpha$")

ax.set_ylabel("score")

ax.set_ylim(0,1.0)

ax.set_title(r"多项式朴素贝叶斯参数$\alpha$与分类得分",fontproperties = myfont)

ax.set_xscale("log")
```

```
ax.legend(loc="best", framealpha=0.5)

plt.show()

MultinomialNB_alpha(x_train, x_test, y_train, y_test)


# 深度学习

# 将 target 转换为整型

y_train_int = []

y_test_int = []

for i in y_train:

    y_train_int.append(int(i))

for j in y_test:

    y_test_int.append(int(j))

# 每行数据 10 个特征，都是 real-value 的

feature_columns = [ts.contrib.layers.real_valued_column("",
dimension=10)]

# 构建一个 DNN 分类器，3 层，其中每个隐含层的节点数量分别为
10, 20, 10，目标的分类 3 个，并且指定了保存位置
```

```
node_num =
```

```
[[2,2],[2,5],[2,2,2],[2,3,2],[2,5,2],[3,5,3],[5,10,5],[8,12,8],[10,20,10],[15,20,15],[20,25,20],[25,30,25],[25,30,30],[30,35,30]]
```

```
num_ids = []
```

```
train_accuracies = []
```

```
test_accuracies = []
```

```
pred_test = []
```

```
for i in node_num:
```

```
    classifier =
```

```
ts.contrib.learn.DNNClassifier(feature_columns=feature_columns,hidden  
_units=i,n_classes=2)
```

```
    classifier.fit(x=x_train, y=y_train_int,steps=5000)
```

```
    train_accuracy =
```

```
classifier.evaluate(x=x_train,y=y_train_int)["accuracy"]
```

```
    train_accuracies.append(train_accuracy)
```

```
    test_accuracy = classifier.evaluate(x=x_test,  
y=y_test_int)["accuracy"]
```

```
test_accuracies.append(test_accuracy)

num_id = node_num.index(i)

num_ids.append(num_id)

print(num_id,":",i)

print()

pred_tests = classifier.predict(x=x_test,as_iterable = True)

for value in pred_tests:

    pred_test.append(value)

# print(classification_report(y_test_int,pred_test))

fig = plt.figure()

ax = fig.add_subplot(1,1,1)

ax.plot(num_ids,train_accuracies,label = "Training accuracy")

ax.plot(num_ids,test_accuracies,label = "Testing accuracy")

ax.set_xlabel(r"id")

ax.set_ylabel("accuracy")
```

```
ax.set_ylim(0,1.0)
```

```
ax.set_title(r"深度神经网络与分类准确率",fontproperties = myfont)
```

```
ax.legend(loc="best", framealpha=0.5)
```

```
plt.show()
```

```
# 隐藏节点数最优为[25,30,25]
```

```
# 训练最终模型并集成结果
```

```
data_predict = pd.read_csv("result_use.csv")[1328:]
```

```
# print(data_predict.head(5))
```

```
temp_predict = vector.transform(data_predict['comment']).todense()
```

```
y_precict_real = data_predict['target']
```

```
x_predict = temp_predict
```

```
print(x_predict)
```

```
# print(y_precict_real)
```

```
# SVM
```

```
clf_svm = svm.SVC(kernel = 'sigmoid',gamma=0.020236,  
coef0=0,probability=True)
```

```
clf_svm.fit(x_train,y_train)
```

```
pre_svm = clf_svm.predict_proba(x_predict)
```

```
# print(pre_svm)
```

```
f = open("pre_svm_p.txt","a")
```

```
for i in pre_svm:
```

```
    f.write(str(i)+"\n")
```

```
f.close()
```

```
## 朴素贝叶斯
```

```
## 将 VSM 每个值加 1
```

```
for i in range(temp.shape[0]):
```

```
    temp[i] = temp[i] + 1
```

```
    y = data['target']
```

```
    x = temp
```

```
    x_train, x_test, y_train, y_test = cross_validation.train_test_split(x, y,  
test_size=0.3, random_state=1)
```

```
for i in range(temp_predict.shape[0]):  
    temp_predict[i] = temp_predict[i] + 1
```

```
clf_nb = MultinomialNB(alpha=0.01)
```

```
clf_nb.fit(x_train,y_train)
```

```
pre_nb = clf_nb.predict_proba(x_predict)
```

```
f = open("pre_nb_p.txt","a")
```

```
for i in pre_nb:
```

```
    f.write(str(i) + "\n")
```

```
f.close()
```

```
## DNN
```

```
y_train_int = []
```

```
for i in y_train:
```

```
    y_train_int.append(int(i))
```

```
feature_columns = [ts.contrib.layers.real_valued_column("",
dimension=10)]

clf_dnn =
ts.contrib.learn.DNNClassifier(feature_columns=feature_columns,hidden
_units=[25,30,25],n_classes=2)

clf_dnn.fit(x=x_train, y=y_train_int,steps=5000)

pre_dnn0 = clf_dnn.predict_proba(x=x_predict,as_iterable = True)

pre_dnn = []

for value in pre_dnn0:

    pre_dnn.append(value)

f = open("pre_dnn_p.txt","a")

for i in pre_dnn:

    f.write(str(i) + "\n")

f.close()
```